JobSearch-SUBUL

Career Assistant + CV Booster

Technical Documentation & Scenario Guide

Stack	FastAPI · Azure CosmosDB · Next.js 14 · Azure OpenAI GPT-4o-mini · SentenceTransformers
Backend	Python 3.12 — port 8000 (uvicorn)
Frontend	Next.js 14 App Router — port 3000
Database	Azure CosmosDB (users + AgentSearchJobs containers)

v1.0 — March 2026

1. Architecture Overview

JobSearch-SUBUL is a full-stack AI career assistant composed of two independent servers that communicate via REST API and SSE (Server-Sent Events).

System Components

- Backend (FastAPI @ port 8000) — all AI processing, scraping, scoring, DB operations
- Frontend (Next.js @ port 3000) — user interface, proxies all API calls to backend
- Azure CosmosDB — stores user profiles (users container) and matched jobs (AgentSearchJobs container)
- Azure OpenAI GPT-4o-mini — CV structuring, rewriting, job matching explanations
- SentenceTransformers (all-MiniLM-L6-v2) — semantic job-CV matching (cosine similarity)
- Fine-tuned BiEncoder — AI match score 0.0–1.0 (stored as match_score in DB)

Communication Flow

1. User Browser	**2.** Next.js :3000	**3.** FastAPI :8000	**4.** CosmosDB / OpenAI

All browser requests hit Next.js first. The next.config.ts rewrites proxy them transparently to the FastAPI backend. The frontend never calls the backend directly — it always goes through the Next.js proxy layer.

2. User Scenarios

Scenario A — First-time user: Upload CV & Search Jobs

1. /upload-cv	**2.** Extract CV	**3.** Enrich	**4.** Save Profile	**5.** Search Jobs

Step 1 — Navigate to /upload-cv?user_id=7

The user lands on the CV upload page. CVBoosterApp.tsx reads user_id from the URL query string and calls GET /platform-data/{user_id} to load any existing SUBUL labs and certifications.

Step 2 — Upload CV (PDF or DOCX)

The user drags or selects their CV file. The frontend sends:

```
POST /extract-cv   multipart/form-data  { file }
```

The backend (save_cv.py) extracts raw text using pdfplumber or python-docx, detects missing sections (experience, education, languages), and returns:

```
{ "text": "...", "missing_sections": ["languages"] }
```

Step 3 — Enrich with SUBUL data (EnrichScreen)

The EnrichScreen.tsx component displays the user's SUBUL labs, certifications, and quiz results. The user selects which items to include and fills in any missing sections via MissingSectionsModal.

Step 4 — Save to CosmosDB

The user clicks Save & Search. The frontend sends:

```
POST /save-cv   multipart/form-data  { file, user_id, quiz_data, labs_data,
certs_data, extra_data }
```

The backend: (1) extracts CV text, (2) calls GPT-4o-mini to structure the profile into JSON fields, (3) upserts the document into the CosmosDB users container with fields: role, seniority, skills, summary, cv_raw_text, etc.

Step 5 — Redirect to job search

On success, the frontend redirects to /jobs-search?user_id=7, which checks if the user has saved jobs and redirects to /app?user_id=7 (with or without &scan=1).

Scenario B — Returning user: View Matched Jobs

1. /app?user_id=7	**2.** Load Jobs SSE	**3.** Display Cards	**4.** Filter & Sort

Step 1 — Dashboard loads

The Next.js App component calls GET /jobs/{user_id} which returns an SSE stream of all saved jobs from CosmosDB, ordered by match_score DESC.

Step 2 — Job cards render

Each SSE event contains a job object with: title, industry, location, match_score (0.0–1.0), cosine (0.0–1.0), gap_missing (list of missing skills), gap_coverage, xai (LLM explanation object).

Step 3 — Filters

The user can filter by role keyword, location, and minimum AI match score (40% / 55% / 75%). All filtering is client-side on the loaded job array.

Scenario C — New scan: Analyze CV against live job market

1. POST /scan	2. Scrape 10 sources	3. Enrich + Score	4. Save to DB	5. SSE stream

Step 1 — Trigger scan

When the user arrives at /app with &scan=1, the frontend sends:

```
POST /scan   { user_id: 7, cv_raw_text: '...' }
```

Step 2 — Parallel scraping

The scraping_pipeline.py launches 10 scrapers in parallel: aijobs, remoteok, tanitjobs, greenhouse, eluta, whatjobs, indeed, linkedin, lever, wttj. Each returns a list of job listings.

Step 3 — Enrichment & scoring per job

For each job that passes the cosine threshold (≥ 0.60), the pipeline:

- Calls extract_with_llm() or uses source-specific field mapping to get structured job details
- Runs the fine-tuned BiEncoder model via mtch.predict() → match_score 0.0–1.0
- Computes skills gap via mtch.compute_skills_gap() → gap_missing, gap_coverage
- Optionally generates XAI explanation via xai_explainer.explain_job_match()
- Saves the enriched job card to CosmosDB via database.insert_job()
- Emits a 'job' SSE event to the frontend

Step 4 — Frontend renders in real time

Each SSE 'job' event immediately renders a new job card in the dashboard grid. No page reload required.

Scenario D — ATS Score: Compare CV to a specific job

1. Click ATS Score	2. POST /api/ats-score	3. GPT Structure	4. MiniLM Match	5. Show Results

Step 1 — User clicks 'Calculer le Score ATS'

The ATSModal opens and sends:

```
POST /api/ats-score   { user_id: '7', job_description: '...' }
```

The job_description is built from the job card fields: title, experience, skills_req, skills_bon, description.

Step 2 — Backend processing (main.py)

- Loads cv_raw_text from CosmosDB for the user
- Calls ats_extractor.prepare_cv_text() to clean the CV text
- Calls ats_structurer.structure_cv() and structure_jd() in parallel via GPT-4o-mini
- Calls ats_scorer.compute_ats_score() with the MiniLM semantic matcher

Step 3 — Score breakdown returned

The response includes: total_score (0–100), breakdown per dimension (required_skills 55%, preferred_skills 10%, experience 15%, designation 10%, degree 5%, languages 5%), missing_required, missing_preferred, suggestions.

Scenario E — CV Booster: Rewrite CV for a specific job

1. /cv-boost?job_id=...	2. Load Job Details	3. LLM Rewrite	4. Download DOCX

Step 1 — Open CV Booster

From the ATSModal, the user clicks 'Enhance CV for this job'. This opens /cv-boost?user_id=7&job_id=<encoded_url> in a new tab.

Step 2 — JobBoosterApp.tsx loads

The page calls:

```
GET /job-data/<encoded_url>
```

Which decodes the URL, queries CosmosDB by the url field, and returns title, description, requirements, location.

Step 3 — User selects format and clicks Enhance

The frontend sends:

```
POST /enhance-cv-for-job   { user_id, job_id, cv_format }
```

The backend (enhance_cv.py): loads cv_raw_text from CosmosDB, loads job details, injects job context into the LLM prompt, calls GPT-4o-mini to rewrite the CV targeting that specific role, generates a DOCX file in the chosen format (ATS / Basic / Modern / Canadian / Europass).

Step 4 — DOCX download

The backend streams the DOCX with response headers: X-ATS-Score-Before, X-ATS-Score-After, X-Parsed-CV. The frontend triggers a browser download.

Scenario F — Career Chatbot

The chat sidebar (ChatSidebar.tsx) sends messages to:

```
POST /api/chat   { user_id, message, jobs_context: [...] }
```

The chat_router.py builds a system prompt that includes: the user's full profile from CosmosDB, all saved jobs with scores and skills gaps, source index by scraping origin, location index. The LLM (GPT-4o-mini) responds in the user's language (French or English), detected automatically.

3. File & Folder Reference

3.1 Backend — Root (JobSearch-SUBUL/)

File / Folder	Role	Key exports / endpoints
`main.py`	FastAPI app entry point — CORS, startup, route registration, ATS score endpoint, report PDF generation	app, startup(), /api/ats-score, /api/report, /api/market
`database.py`	All CosmosDB CRUD — users container + AgentSearchJobs container	get_user(), upsert_user(), insert_job(), get_jobs_for_user(), save_chat_message()
`scraping_pipeline.py`	CV extraction helpers + main SSE scraping pipeline — 10 sources in parallel	scraping_router, pipeline(), detect_and_translate_cv(), extract_cv_title()
`chat_router.py`	Chatbot endpoint — builds LLM system prompt with user profile + jobs context	chat_router, POST /api/chat, GET /api/chat/history
`jobs_router.py`	Jobs SSE stream, matching API, skills gap, roadmap endpoints	jobs_router, GET /jobs/{user_id}, POST /api/gap, POST /api/roadmap
`enhance_cv.py`	CV rewriting, DOCX generation (5 formats), job-targeted boost endpoint	enhance_router, POST /boost-cv, POST /enhance-cv-for-job, GET /job-data/{job_id}
`save_cv.py`	CV text extraction + save profile to CosmosDB	POST /extract-cv, POST /save-cv
`cv_extraction.py`	PDF/DOCX text + photo extraction utilities	extract_text_from_pdf(), extract_text_from_docx(), extract_photo_from_pdf()
`explain_ats.py`	LLM-generated ATS explanation for CV sections	explain_ats_score()
`db_platform.py`	PostgreSQL connection — loads SUBUL labs + certifications for a user	get_platform_data_or_fallback(), fetch_recommendations()
`matcher.py`	Fine-tuned BiEncoder model — CV × Job scoring, skills gap computation	predict(), compute_skills_gap(), compute_combined_score()
`ats_extractor.py`	CV text cleaning and normalization for ATS scoring	prepare_cv_text()
`ats_structurer.py`	GPT-4o-mini CV + JD structuring into normalized JSON	structure_cv(), structure_jd()
`ats_matcher.py`	MiniLM-L6-v2 semantic skill matching for ATS	match_skills(), semantic_similarity()
`ats_scorer.py`	Multi-dimensional ATS score computation (6 weighted dimensions)	compute_ats_score()
`xai_explainer.py`	LLM-as-judge — generates human-readable score explanations and roadmap courses	explain_job_match(), enrich_roadmap_with_llm()
`job_analyzer_agent.py`	Career analysis agent — market analysis, skills gap, roadmap generation (CLI + API)	MarketAnalysis, CandidateProfile, compute_gap(), generate_roadmap()

File / Folder	Role	Key exports / endpoints
`llm_extractor.py`	Async job page fetcher + GPT-4o extractor (aijobs, remoteok fallback)	extract_with_llm()
`.env`	Environment variables — Azure OpenAI, CosmosDB, SerpAPI keys	AZURE_OPENAI_*, AZURE_COSMOS_*, SERPAPI_API_KEY

3.2 Backend — Scrapers

File / Folder	Role	Key exports / endpoints
`scraper.py`	Facade — re-exports all scraper functions for external use	scrape_aijobs, scrape_remoteok, ...
`scraper_utils.py`	Shared constants + date/text helpers used by all scrapers	MAX_AGE_DAYS, _parse_date(), _age_label(), _too_old()
`scraper_aijobs.py`	aijobs.ai — SerpApi Google Web + HTTP direct fallback	scrape_aijobs()
`scraper_remoteok.py`	remoteok.com — HTML parser + JSON API fallback	scrape_remoteok()
`scraper_tanitjobs.py`	tanitjobs.com — SerpApi Google Web (Tunisia jobs)	scrape_tanitjobs()
`scraper_greenhouse.py`	greenhouse.io — public JSON API for 15 tech company boards	scrape_greenhouse()
`scraper_eluta.py`	eluta.ca — SerpApi Google Web (Canada jobs)	scrape_eluta()
`scraper_whatjobs.py`	whatjobs.com — SerpApi Google Web (global)	scrape_whatjobs()
`scraper_indeed.py`	Indeed — SerpApi Google Jobs API, filters via:indeed links	scrape_indeed()
`scraper_linkedin.py`	LinkedIn — SerpApi Google Jobs API, filters linkedin.com links	scrape_linkedin()
`scraper_lever.py`	Lever.co — public postings JSON API for 20 tech companies	scrape_lever()
`scraper_wttj.py`	Welcome to the Jungle — SerpApi Google organic results	scrape_wttj()
`serpapi_google_web_common.py`	Shared SerpApi urllib client — pagination, deduplication, CSV output	scrape_google_web()

3.3 Frontend — frontend/

File / Folder	Role	Key exports / endpoints
`next.config.ts`	Next.js config — rewrites proxy all /api/*, /scan, /jobs/*, /boost-cv, /enhance-cv-for-job to backend :8000	rewrites()

File / Folder	Role	Key exports / endpoints
`app/layout.tsx`	Root layout — Sora + DM Mono fonts, global CSS, metadata	RootLayout
`app/globals.css`	Global CSS reset and CSS variables	—
`app/page.tsx`	Root page — redirects to /jobs-search	LoginPage
`app/jobs-search/page.tsx`	Auth check — calls /api/user/{user_id}/has_jobs and redirects to /app with or without &scan=1	JobsSearchPage
`app/upload-cv/page.tsx`	CV upload entry point — renders CVBoosterApp	UploadCVPage
`app/cv-boost/page.tsx`	CV-for-job booster page — renders JobBoosterApp	CvBoostPage
`app/app/page.tsx`	Main dashboard — Matches, Skills Gap, Roadmap, Market, Report tabs + chat sidebar	Dashboard

3.4 Frontend — CV Booster Components (app/app/components/cv-booster/)

File / Folder	Role	Key exports / endpoints
`CVBoosterApp.tsx`	Main CV upload wizard — file drop, platform data fetch, 5-phase state machine (idle→scanning→enrich→saving→done)	default export
`CVBoosterApp.css`	Full dark-theme CSS for CVBoosterApp — blobs, stepper, dropzone, loading/error screens	—
`EnrichScreen.tsx`	Step 2 of upload wizard — select labs, certs, quiz; fill missing sections via modal; Save & Search button	EnrichScreen, PLATFORM_LABS, PLATFORM_CERTS, DEFAULT_SELECTIONS
`ExplainScreen.tsx`	ATS score breakdown visualization — radar-style dimension bars, keyword matches, suggestions	ExplainScreen
`FormatScreen.tsx`	DOCX format selector for /boost-cv flow — 5 format previews (ATS, Basic, Modern, Canadian, Europass)	FormatScreen
`MissingSectionsModal.tsx`	Modal for adding missing CV sections (education, experience, languages) before saving	MissingSectionsModal
`ResultInsights.tsx`	ATS score results display — before/after score, keyword breakdown, suggestions	ResultInsights

3.5 Frontend — CV-for-Job Booster (app/cv-boost/)

File / Folder	Role	Key exports / endpoints
`JobBoosterApp.tsx`	Main component for job-targeted CV enhancement — fetches job details, shows format selector, triggers /enhance-cv-for-job, handles	JobBoosterApp

File / Folder	Role	Key exports / endpoints
	DOCX download + score display	
`FormatScreen.tsx`	Format selector (copy of cv-booster version, scoped to job boost flow)	FormatScreen
`page.tsx`	Next.js page wrapper — renders JobBoosterApp with user_id + job_id from query params	CvBoostPage

4. Data Flow & Key Data Structures

4.1 CosmosDB — users container

Document structure (partition key: /id)

- `id: "7"  (user_id as string)`
- `role, seniority, years_exp, industry, education, skills (CSV)`
- `summary, bullets, cv_raw_text`
- `first_name, last_name, email, linkedin`
- `ats_score_before, ats_score_after`

4.2 CosmosDB — AgentSearchJobs container

Document structure (partition key: /url)

- `id, id_job (UUID), user_id (int), url, source, title, industry`
- `match_score (float 0.0-1.0), cosine_score (float), combined_score (float)`
- `skills_gap (JSON string of missing skills list)`
- `must_have, nice_to_have, description, location, salary, contract, remote`
- `xai (object: interpretation, score_formula, explanations, tip, strength)`
- `dedupe_key (normalized title|company for deduplication)`

4.3 Score Definitions

File / Folder	Role	Key exports / endpoints
`match_score`	AI BiEncoder (fine-tuned all-MiniLM-L6-v2) — 0.0 to 1.0	Primary score shown in dashboard
`cosine_score`	Cosine similarity between CV title embedding and job title embedding — 0.0 to 1.0	Secondary filter (threshold 0.60)
`combined_score`	match_score × √(skills_coverage) — penalizes large skills gaps	Used in report & PDF
`ATS total`	Weighted sum across 6 dimensions — 0 to 100	Shown in ATSModal

5. API Endpoints Quick Reference

5.1 CV & Profile

File / Folder	Role	Key exports / endpoints
`POST /extract-cv`	Extract text from uploaded PDF/DOCX. Returns raw text + missing sections.	save_cv.py
`POST /save-cv`	Structure CV with GPT + upsert to CosmosDB users container.	save_cv.py
`GET /api/user/{user_id}`	Check if user exists in CosmosDB and has saved jobs.	main.py
`GET /api/profile`	Load user profile fields from CosmosDB.	main.py
`GET /platform-data/{uid}`	Load SUBUL labs, certs, quiz from PostgreSQL.	enhance_cv.py

5.2 Jobs & Scan

File / Folder	Role	Key exports / endpoints
`POST /scan`	Start full CV scan — scrape 10 sources, score, save to DB. Returns SSE stream.	scraping_pipeline.py
`GET /jobs/{user_id}`	Stream saved jobs from CosmosDB as SSE. Ordered by match_score DESC.	jobs_router.py
`GET /api/matches/{uid}`	REST version of job list (for dashboard Next.js direct calls).	jobs_router.py
`POST /api/gap`	Compute skills gap between user profile and market demand.	jobs_router.py
`POST /api/roadmap`	Generate phased learning roadmap for missing skills.	jobs_router.py
`GET /api/market`	Market analytics — top skills, companies, locations, salary stats.	main.py

5.3 ATS & CV Booster

File / Folder	Role	Key exports / endpoints
`POST /api/ats-score`	Compute ATS score between user CV and a job description. Returns 0–100 + breakdown.	main.py
`POST /boost-cv`	Full CV enhancement — upload file, LLM rewrite, DOCX download.	enhance_cv.py
`POST /enhance-cv-for-job`	Job-targeted CV rewrite — loads CV from CosmosDB, rewrites for specific job.	enhance_cv.py

File / Folder	Role	Key exports / endpoints
`GET /job-data/{job_id:path}`	Fetch job details from CosmosDB by URL (used by cv-boost page).	enhance_cv.py
`POST /apply-format`	Re-generate DOCX in a different format from parsed CV JSON.	enhance_cv.py

5.4 Chat & Report

File / Folder	Role	Key exports / endpoints
`POST /api/chat`	Chatbot — builds LLM prompt with user profile + jobs context, responds in FR/EN.	chat_router.py
`GET /api/chat/history`	Load chat history from CosmosDB for a user.	chat_router.py
`POST /api/report`	Generate Markdown career report (profile + gaps + matches).	main.py
`POST /api/report/pdf`	Generate PDF career report using reportlab.	main.py

6. Environment Variables (.env)

File / Folder	Role	Key exports / endpoints
`AZURE_OPENAI_ENDPOINT`	Azure OpenAI resource endpoint URL	Required
`AZURE_OPENAI_API_KEY`	Azure OpenAI API key	Required
`AZURE_OPENAI_API_VERSION`	API version (default: 2024-08-01-preview)	Required
`AZURE_OPENAI_DEPLOYMENT_NAME`	GPT deployment name (e.g. gpt-4o-mini)	Required
`AZURE_COSMOS_ENDPOINT`	CosmosDB account endpoint URL	Required
`AZURE_COSMOS_KEY`	CosmosDB primary key	Required
`AZURE_COSMOS_DATABASE_NAME`	Database name (e.g. EduTech_AI_Production)	Required
`AZURE_COSMOS_QUIZ_CONTAINER_NAME`	Jobs container name (AgentSearchJobs)	Required
`AZURE_COSMOS_USERS_CONTAINER`	Users container name (users)	Required
`SERPAPI_API_KEY`	SerpApi key for Google Search scrapers (aijobs, eluta, etc.)	Required for scraping
`AZURE_HOST / AZURE_DB / AZURE_USER / AZURE_PASSWORD / AZURE_PORT`	PostgreSQL connection for SUBUL platform data (labs, certs)	Required for enrichment
`EXPLAINABLE_AI_ENABLED`	Set to 1 to enable LLM XAI explanations and roadmap courses	Optional (default: auto)
`DEEPGRAM_API_KEY`	Deepgram STT/TTS key (voice feature — disabled by default)	Optional

7. Startup & Development

7.1 Backend

```
cd JobSearch-SUBUL
python -m venv venv && venv\Scripts\activate   # Windows
pip install -r requirements.txt
uvicorn main:app --reload --port 8000
```

On startup, main.py: initializes CosmosDB containers, loads MarketAnalysis from file-based jobs, then refreshes from CosmosDB via _refresh_market_analysis().

7.2 Frontend

```
cd frontend
npm install
npm run dev   # starts on port 3000
```

All API calls are proxied via next.config.ts rewrites. The frontend never needs to know the backend port — it always calls /api/*, /scan, /jobs/*, etc. which Next.js forwards to http://localhost:8000.

7.3 Required Python packages (key)

- fastapi, uvicorn — web server
- openai — Azure OpenAI client
- sentence-transformers — MiniLM embeddings
- azure-cosmos — CosmosDB client
- python-docx — DOCX generation
- reportlab — PDF report generation
- aiohttp, beautifulsoup4 — async scraping
- python-dotenv — .env loading
- asyncpg (or psycopg2) — PostgreSQL for SUBUL platform data